Ausgabe am **26.05.2025**
Abgabe bis zum **16.06.2025, 13:30 Uhr**

Hochschule Fulda
University of Applied Sciences

Prof. Dr. Thomas Wiemann

Robotik
Übungsblatt 4

Sommersemester 2025

. Aufgabe A4.1 (Kalman Filter - (Theorie)) (25 Punkte)

In der Vorlesung haben Sie den Kalman-Filter als Spezialfall des Bayes-Filters kennengelernt. Die Mathematik des Kalman-Filters lässt sich wie folgt zusammenfassen:

$$\mu_{t+1} = A\mu_t + Bu_t + K_{t+1}(z_{t+1} - H(A\mu_t + Bu_t)) \tag{1}$$
$$\Sigma_{t+1} = (\mathbb{1} - K_{t+1}H)(A\Sigma_t A^T + \Sigma_u) \tag{2}$$
$$K_{t+1} = (A\Sigma_t A^T + \Sigma_u)H^T(H(A\Sigma_t A^T + \Sigma_u)H^T + \Sigma_z)^{-1} \tag{3}$$

(a) Wie wird der Zustand x_t zum Zeitpunkt t im Kalman-Filter modelliert? Welche Annahme über die Verteilung von x_t liegt der Modellierung zugrunde?

(b) Was modellieren die Matrizen A, B und H?

(c) Welche Bedeutung hat die Matrix K_{t+1}?

Aufgabe A4.2 (Kalman Filter (Praxis)) (25 Punkte)

Im Package `Kalman Example` findet sich der Code für ein simples Beispiel eines Kalman Filters. Es wird eine fallende Kugel simuliert. Wenn Sie RViz2 starten, können Sie nach dem Start der Node `kalman_example` aus dem gleichnamigen Paket die Position der Kugel im Frame `world` visualisieren. Fügen Sie dazu das Topic `object` vom Typ `PointStamped` hinzu. Im Terminal der `kalman_example`-Node sehen Sie die Ausgabe der Iterationsnummer sowie den geschätzten und korrigierten Weltzustand. Inspizieren Sie den Code in der Quelldatei `KalmanExample.cpp` und beantworten Sie folgende Fragen zum Quellcode und der Konsolenausgabe:

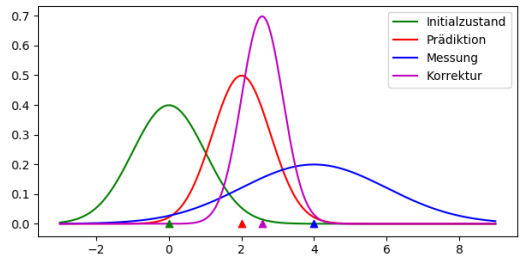
(a) Erklären Sie die Modellierung des freien Falls und des Auftreffens der Kugel auf dem "Boden"

(b) Interpretieren Sie die Konsolenausgabe. Warum entwickeln sich die Werte der Prädiktion und Korrektur wie beobachtet?

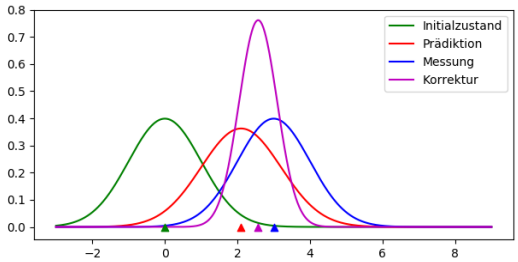
(c) Warum strebt die geschätzte Beschleunigung gegen Null und wie kommt dieser Effekt zustande?

Aufgabe B4.1 (1D Kalman Filter (Theorie)) (25 Punkte)

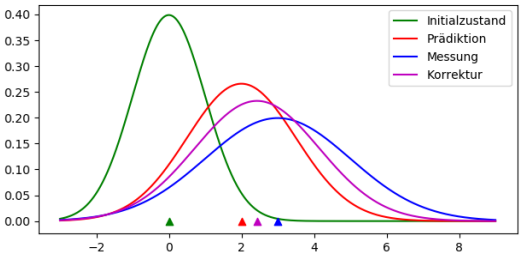
(a) Finden Sie heraus welche der vorliegenden Ergebnisse aus einem 1D-Kalman-Filter stammen und welche nicht. Begründen Sie Ihre Antworten.



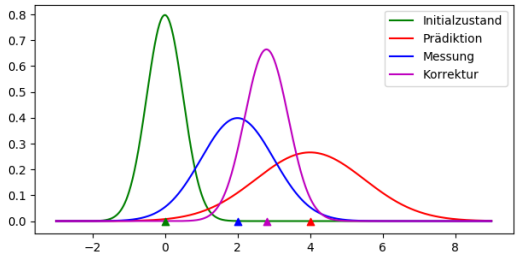
(a)



(b)



(c)



(d)

Aufgabe B4.2 (EKF / Robot Localization (Praxis)) **(25 Punkte)**

Lineare Kalman-Filter sind unter folgenden Bedingungen optimal:

- Alle Sensoren sind perfekt modelliert
- Das System kann komplett durch lineare Zustandsübergänge beschrieben werden
- Das Sensorrauschen ist normalverteilt
- Der geschätzte Zustand ist normalverteilt und unimodal

In der Praxis sind diese Annahmen in der Regel nicht erfüllt. Beschreiben Sie mögliche Abweichungen von diesen Annahmen bei der Nutzung des RosBots.

Ab einem bestimmten Punkten machen lineare Kalman-Filter keinen Sinn mehr und man nutzt *Extended Kalman Filter (EKF)* oder *Unscented Kalman Filters (UKF)*. In ROS2 gibt es dazu das Paket "Robot Localization". In diesem können Sie eine Implementierung eines EKFs nutzen. Dazu müssen Sie die Topics der Sensoren auf den RosBots passend mappen. Ein Tutorial für Robot Localization finden Sie unter `https://automaticaddison.com/sensor-fusion-using-the-robot-localization-package-ros-2/`. Die offizielle Dokumentation finden Sie auf `ros.org`.

Setzen Sie einen ROS2-Workspace auf, der das Robot Localization Package nutzt, um die Odometrie und IMU-Daten des RosBots zu filtern. Nutzen Sie dazu des `cmd_vel`-Topic als Prior und die IMU und Odometrie-Werte als Messungen / Posteriors. Parametrisieren Sie das Package passend und publishen Sie die gefilderten Daten auf einem neuen Topic. Hinweis: Oft werden die Linearbeschleunigungen der IMU bei der Berechnung der Translationskomponenten komplett ignoriert.

Schreiben Sie ein Launchfile und ein `.yaml`-File mit den passenden Parametern. Spielen Sie ein wenig mit den Werten, um ein Gefühl für den Einfluss auf das Endergebnis zu bekommen. Vergleichen Sie die mit den von Ihnen gefundenen besten Parametern gefilterte Positionsschätzung mit der ungefilterten Odometrie und den IMU-Trajektorieren.